Agenda

- Inheritance
  - Shadowing
  - Object Slicing
  - Virtual Function
  - Pure Virtual Function and Abstract Class
- Static Data Member and Member Function
- Exception Handling
- Template

---

1. Inheritance

Inheritance allows a derived class to acquire properties and behaviors of a base class, enabling code reuse and hierarchical classification.

1.1 Shadowing (Function Hiding)

- When a derived class defines a function with the **same name** as a base class function (but without `virtual`), the base class version is **hidden**.
- This is different from overriding — it hides all overloaded versions of that function in the base class.
- Access the hidden base function using the scope resolution operator: `Base::functionName()`.

```
class Base {
public:
    void show() { cout << "Base show()" << endl; }
};

class Derived : public Base {
public:
    void show() { cout << "Derived show()" << endl; } // shadows Base::show()
};

Derived d;
d.show();        // Derived show()
d.Base::show();  // Base show()
```

1.2 Object Slicing

- Occurs when a **derived class object is assigned to a base class object** (by value).
- The extra members added by the derived class are "sliced off" — only the base part is copied.
- Slicing does **not** happen with pointers or references.

```
class Base { public: int x; };
class Derived : public Base { public: int y; }; // extra member

Derived d; d.x = 10; d.y = 20;
Base b = d;  // y is sliced off — only x is copied
```

> **Tip:** Always use pointers (`Base*`) or references (`Base&`) when working with polymorphism to avoid slicing.

1.3 Virtual Function

- Declared with the `virtual` keyword in the base class.
- Enables **runtime polymorphism** — the correct function is called based on the **actual object type**, not the pointer/reference type.
- Mechanism: compiler uses a **vtable** (virtual table) per class and a **vptr** (virtual pointer) per object.

```
class Animal {
public:
    virtual void sound() { cout << "Generic sound" << endl; }
};

class Dog : public Animal {
public:
    void sound() override { cout << "Woof!" << endl; }
};

Animal* a = new Dog();
a->sound(); // Woof! — resolved at runtime via vtable
```

- Use `override` keyword in derived class for safety (compiler checks).
- Use `virtual` destructor in base class when deleting via base pointer.

1.4 Pure Virtual Function and Abstract Class

- A **pure virtual function** is declared with `= 0` — it has no implementation in the base class.
- A class containing at least one pure virtual function is an **abstract class**.
- Abstract classes **cannot be instantiated** directly.
- Derived classes **must override** all pure virtual functions to become concrete (instantiable).

```
class Shape {  // Abstract class
public:
    virtual double area() = 0;  // pure virtual
    virtual void draw() = 0;    // pure virtual
};
```

```
class Circle : public Shape {
    double r;
public:
    Circle(double r) : r(r) {}
    double area() override { return 3.14 * r * r; }
    void draw() override { cout << "Drawing Circle" << endl; }
};

// Shape s;      // Error: cannot instantiate abstract class
Shape* s = new Circle(5); // OK
```

---

2. Static Data Member and Member Function

Static Data Member

- Shared across **all objects** of the class — only one copy exists.
- Declared inside the class with `static`, defined **outside** the class.
- Exists even before any object is created.

```
class Counter {
public:
    static int count;  // declaration
    Counter() { count++; }
};

int Counter::count = 0;  // definition (mandatory outside)

Counter c1, c2, c3;
cout << Counter::count; // 3
```

Static Member Function

- Can be called **without an object** using `ClassName::function()`.
- Can only access **static data members** — has no `this` pointer.

```
class Counter {
    static int count;
public:
    Counter() { count++; }
    static int getCount() { return count; } // static member function
};

cout << Counter::getCount(); // called without object
```

---

3. Exception Handling

Provides a structured way to handle **runtime errors** without crashing the program.

Keywords

Keyword	Purpose
`try`	Block of code that may throw an exception
`throw`	Raises (throws) an exception
`catch`	Catches and handles the exception

Syntax

```
try {
    // risky code
    throw <expression>;  // can throw int, string, object, etc.
}
catch (type var) {
    // handle exception
}
catch (...) {
    // catch-all handler (catches any type)
}
```

Example

```
int divide(int a, int b) {
    if (b == 0) throw b;
    return a / b;
}

try {
    cout << divide(10, 0);
}
catch (int e) {
    cout << "Error: " << e << endl;
}
```

Key Points

- Multiple `catch` blocks can follow one `try` block.
- `catch(...)` must be the **last** catch block.
- Exceptions propagate up the call stack until caught.
- Custom exception classes can inherit from `std::exception`.

---

4. Templates

Templates enable **generic programming** — write code that works with any data type.

Function Template

```
template <typename T>
T maxVal(T a, T b) {
    return (a > b) ? a : b;
}

cout << maxVal(3, 7);         // int version
cout << maxVal(3.5, 2.1);     // double version
cout << maxVal('a', 'z');     // char version
```

Class Template

```
template <typename T>
class Stack {
    T arr[100];
    int top = -1;
public:
    void push(T val) { arr[++top] = val; }
    T pop() { return arr[top--]; }
    bool isEmpty() { return top == -1; }
};

Stack<int> s1;
```

Key Points

- `typename` and `class` are interchangeable in template parameter lists.
- Template instantiation happens at **compile time**.
- You can have **multiple template parameters**: `template <typename T, typename U>`.
- **Template specialization** allows custom behavior for specific types.
